

COMPILATION — S7

Analyse syntaxique LL

Grammaires LL(1) — PREMIER, SUIVANT, table d'analyse

Fiche de cours + exercices corrigés — Préparation M1 ILIA

1. Principe de l'analyse descendante

L'analyse syntaxique **LL** (Left-to-right scan, Leftmost derivation) construit une dérivation gauche de la chaîne d'entrée en partant de l'axiome de la grammaire, en lisant l'entrée de gauche à droite.

Le « L » de LL(k) désigne le sens de lecture de l'entrée (Left to right), le second « L » désigne le type de dérivation (Leftmost), et k désigne le nombre de symboles d'anticipation (*lookahead*) utilisés pour choisir la règle à appliquer. En pratique on étudie surtout le cas **LL(1)** : une seule lettre d'anticipation.

■ *Analyse descendante = on part de l'axiome et on "devine" quelle règle utiliser pour produire la chaîne d'entrée, contrairement à l'analyse ascendante (LR) qui part des terminaux et reconstruit vers l'axiome.*

2. Conditions pour qu'une grammaire soit LL(1)

Une grammaire doit vérifier trois propriétés avant de pouvoir être analysée en LL(1) :

- **Non ambiguë** : chaque chaîne du langage a une unique dérivation gauche.
- **Sans récursivité à gauche** : aucune règle $A \rightarrow A\alpha$ (directe ou indirecte), car l'analyseur descendant boucle sinon indéfiniment.
- **Factorisée à gauche** : si plusieurs règles pour A commencent par le même préfixe, il faut les regrouper pour pouvoir choisir avec un seul symbole d'anticipation.

2.1 Élimination de la récursivité à gauche

Règle générale : si $A \rightarrow A\alpha \mid \beta$ (avec β ne commençant pas par A), on réécrit :

```
A  -> beta A'
A' -> alpha A' | epsilon
```

Exemple : $E \rightarrow E + T \mid T$ se transforme en :

```
E  -> T E'
E' -> + T E' | epsilon
```

2.2 Factorisation à gauche

Si $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$, on factorise le préfixe commun α :

```
A  -> alpha A'
A' -> beta1 | beta2
```

Exemple : $\text{Instr} \rightarrow \text{if Cond then Bloc} \mid \text{if Cond then Bloc else Bloc}$ devient :

```
Instr  -> if Cond then Bloc Instr'
Instr' -> else Bloc | epsilon
```

3. Ensemble PREMIER (FIRST)

PREMIER(X) est l'ensemble des terminaux (et éventuellement ϵ) qui peuvent apparaître en première position d'une chaîne dérivée de X.

Règles de calcul

- Si X est un terminal : $\text{PREMIER}(X) = \{ X \}$.
- Si $X \rightarrow \epsilon$ est une règle : $\epsilon \in \text{PREMIER}(X)$.
- Si $X \rightarrow Y_1 Y_2 \dots Y_n$: on ajoute $\text{PREMIER}(Y_1) \setminus \{\epsilon\}$ à $\text{PREMIER}(X)$. Si $\epsilon \in \text{PREMIER}(Y_1)$, on ajoute aussi $\text{PREMIER}(Y_2) \setminus \{\epsilon\}$, et ainsi de suite. Si tous les Y_i peuvent dériver ϵ , alors $\epsilon \in \text{PREMIER}(X)$.

$\text{PREMIER}(\alpha)$ pour une suite de symboles $\alpha = X_1 \dots X_n$ se calcule avec la même logique de propagation « tant que le symbole précédent peut disparaître (dériver ϵ) ».

4. Ensemble SUIVANT (FOLLOW)

SUIVANT(A) est l'ensemble des terminaux qui peuvent apparaître juste après A dans une forme phrase dérivée de l'axiome. Il ne se calcule que pour les non-terminaux.

Règles de calcul

- \$ (marqueur de fin) $\in \text{SUIVANT}(\text{axiome})$.
- Si $B \rightarrow \alpha A \beta$ (β non vide) : $\text{PREMIER}(\beta) \setminus \{\epsilon\} \subseteq \text{SUIVANT}(A)$.
- Si $B \rightarrow \alpha A$ (A en dernière position), ou $B \rightarrow \alpha A \beta$ avec $\epsilon \in \text{PREMIER}(\beta)$: $\text{SUIVANT}(B) \subseteq \text{SUIVANT}(A)$.

■ On calcule PREMIER pour tous les symboles, puis SUIVANT en itérant sur toutes les règles jusqu'à ce que plus aucun ensemble ne change (point fixe).

5. Exemple guidé

Grammaire des expressions arithmétiques (déjà sans récursivité à gauche) :

```
E  -> T E'
E' -> + T E' | epsilon
T  -> F T'
T' -> * F T' | epsilon
F  -> ( E ) | id
```

PREMIER

Symbole	PREMIER
F	{ (, id }
T'	{ * , epsilon }
T	{ (, id }
E'	{ + , epsilon }
E	{ (, id }

SUIVANT

Non-terminal	SUIVANT
E	{ \$,) }
E'	{ \$,) }
T	{ + , \$,) }
T'	{ + , \$,) }
F	{ * , + , \$,) }

6. Construction de la table d'analyse LL(1)

Pour chaque règle $A \rightarrow \alpha$ de la grammaire :

- Pour chaque terminal $a \in \text{PREMIER}(\alpha)$: $\text{Table}[A, a] = \alpha$.
- Si $\epsilon \in \text{PREMIER}(\alpha)$: pour chaque $b \in \text{SUIVANT}(A)$, $\text{Table}[A, b] = \alpha$ (y compris $\$$ si $\$ \in \text{SUIVANT}(A)$).

Grammaire non LL(1) si une case de la table reçoit plus d'une règle : il faut alors revoir la grammaire (factorisation, récursivité) ou changer de méthode d'analyse (LL(k), LR...).

Table pour l'exemple précédent

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \text{eps}$	$E' \rightarrow \text{eps}$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \text{eps}$	$T' \rightarrow *FT'$		$T' \rightarrow \text{eps}$	$T' \rightarrow \text{eps}$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

7. Algorithme d'analyse par pile

L'analyseur LL(1) utilise une pile initialisée à [Axiome, \$] et l'entrée terminée par \$.

```
pile <- [Axiome, $]
entree <- w $
tant que sommet(pile) != $ :
  X <- sommet(pile)
  a <- symbole courant de l'entree
  si X est terminal :
    si X == a : depiler(pile) ; avancer(entree)
    sinon : ERREUR
  sinon (X non-terminal) :
    si Table[X, a] = X -> Y1 Y2 ... Yn existe :
      depiler(pile)
      empiler Yn, ..., Y2, Y1 (Y1 au sommet)
    sinon : ERREUR
si a == $ : ACCEPTE
```

■ On empile les symboles du membre droit dans l'ordre inverse, pour que Y1 (le premier symbole de la règle) se retrouve au sommet de la pile.

Trace pour l'entrée : id + id * id \$

Pile	Entrée	Action
E \$	id+id*id\$	E -> T E'
T E' \$	id+id*id\$	T -> F T'
F T' E' \$	id+id*id\$	F -> id
id T' E' \$	id+id*id\$	match id
T' E' \$	+id*id\$	T' -> eps
E' \$	+id*id\$	E' -> + T E'
+ T E' \$	+id*id\$	match +
T E' \$	id*id\$	T -> F T'
F T' E' \$	id*id\$	F -> id
id T' E' \$	id*id\$	match id
T' E' \$	*id\$	T' -> * F T'
* F T' E' \$	*id\$	match *
F T' E' \$	id\$	F -> id
id T' E' \$	id\$	match id
T' E' \$	\$	T' -> eps
E' \$	\$	E' -> eps
\$	\$	ACCEPTE

8. Exercices

Exercice 1

Éliminer la récursivité à gauche puis factoriser si nécessaire, pour la grammaire suivante :

$$S \rightarrow S a \mid S b \mid c$$

Donner la grammaire transformée.

Exercice 2

Soit la grammaire :

$$A \rightarrow B c \mid B d$$
$$B \rightarrow e \mid f B$$

Cette grammaire nécessite-t-elle une factorisation à gauche ? Si oui, transformez-la.

Exercice 3

Pour la grammaire suivante, calculer PREMIER et SUIVANT de chaque non-terminal :

$$S \rightarrow A B$$
$$A \rightarrow a A \mid \text{epsilon}$$
$$B \rightarrow b B \mid c$$

Exercice 4

Construire la table d'analyse LL(1) pour la grammaire de l'exercice 3. La grammaire est-elle LL(1) ?

Exercice 5

Grammaire :

$$S \rightarrow i E t S S' \mid a$$
$$S' \rightarrow e S \mid \text{epsilon}$$
$$E \rightarrow b$$

Calculer PREMIER(S), PREMIER(S'), SUIVANT(S), SUIVANT(S'). Cette grammaire est-elle LL(1) ?

Justifier (indice : cas du "dangling else").

Exercice 6

En utilisant la table LL(1) construite en section 6 (grammaire des expressions), tracer pas à pas l'analyse de l'entrée : $(id + id) * id \$$

9. Corrections

Exercice 1

Correction

$S \rightarrow S a \mid S b \mid c$ contient une récursivité à gauche directe ($S \rightarrow S a$ et $S \rightarrow S b$), avec $\beta = c$ (unique alternative ne commençant pas par S).

Application du schéma $A \rightarrow A\alpha \mid \beta \blacksquare A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \varepsilon$:

```
S  -> c S'
S' -> a S' | b S' | epsilon
```

Il n'y a pas de préfixe commun entre les alternatives restantes : pas de factorisation supplémentaire nécessaire.

Exercice 2

Correction

$A \rightarrow B c \mid B d$: les deux alternatives commencent par le même symbole B (préfixe commun). Une factorisation à gauche est donc nécessaire :

```
A  -> B A'
A' -> c | d
```

$B \rightarrow e \mid f B$ n'a pas de préfixe commun entre « e » et « $f B$ » : pas de factorisation nécessaire pour B .

Exercice 3

Correction

PREMIER : $\text{PREMIER}(A) = \{ a, \text{epsilon} \}$ ($A \rightarrow aA$ donne a , $A \rightarrow \varepsilon$ donne ε). $\text{PREMIER}(B) = \{ b, c \}$ ($B \rightarrow bB$ donne b , $B \rightarrow c$ donne c). $\text{PREMIER}(S) = \text{PREMIER}(A) \setminus \{ \varepsilon \} \cup \text{PREMIER}(B)$ car $\varepsilon \in \text{PREMIER}(A)$, donc $\text{PREMIER}(S) = \{ a, b, c \}$.

SUIVANT : $\text{SUIVANT}(S) = \{ \$ \}$ (axiome). Pour A dans $S \rightarrow A B$: $\text{SUIVANT}(A) \supseteq \text{PREMIER}(B) \setminus \{ \varepsilon \} = \{ b, c \}$. Pour B dans $S \rightarrow A B$ (B en dernière position) : $\text{SUIVANT}(B) \supseteq \text{SUIVANT}(S) = \{ \$ \}$. Dans $B \rightarrow b B$, B en dernière position : $\text{SUIVANT}(B) \supseteq \text{SUIVANT}(B)$ (rien de nouveau). Résultat : $\text{SUIVANT}(A) = \{ b, c \}$, $\text{SUIVANT}(B) = \{ \$ \}$.

Exercice 4

Correction

Règles à placer : $S \rightarrow A B$ ($\text{PREMIER} = \{ a, b, c \}$) ; $A \rightarrow aA$ ($\text{PREMIER} = \{ a \}$) ; $A \rightarrow \varepsilon$ ($\text{SUIVANT}(A) = \{ b, c \}$) ; $B \rightarrow bB$ ($\text{PREMIER} = \{ b \}$) ; $B \rightarrow c$ ($\text{PREMIER} = \{ c \}$).

	a	b	c	\$
S	S->AB	S->AB	S->AB	
A	A->aA	A->eps	A->eps	
B		B->bB	B->c	

Aucune case ne contient deux règles : **la grammaire est LL(1).**

Exercice 5

Correction

$\text{PREMIER}(S) = \{ i, a \}$ ($S \rightarrow iEtSS'$ donne i ; $S \rightarrow a$ donne a).

$\text{PREMIER}(S') = \{ e, \text{epsilon} \}$ ($S' \rightarrow eS$ donne e ; $S' \rightarrow \varepsilon$).

SUIVANT(S) : S apparaît dans $S \rightarrow i \text{ E t } S \text{ S'}$ (suivi de S', donc $\text{PREMIER}(S') \setminus \{\epsilon\} = \{e\} \subseteq \text{SUIVANT}(S)$) et comme $\epsilon \in \text{PREMIER}(S')$, SUIVANT(S) hérite aussi de SUIVANT du membre englobant. Comme S est l'axiome, $\$ \in \text{SUIVANT}(S)$. Donc $\text{SUIVANT}(S) = \{e, \$\}$.

$\text{SUIVANT}(S') = \text{SUIVANT}(S) = \{e, \$\}$ (S' est en dernière position dans la règle de S).

Problème : pour construire $\text{Table}[S', e]$, on regarde $\text{PREMIER}(eS) = \{e\} \rightarrow \text{règle } S' \rightarrow eS$. Mais comme $\epsilon \in \text{PREMIER}(S')$ et $e \in \text{SUIVANT}(S')$, la règle $S' \rightarrow \epsilon$ revendique AUSSI la case (S', e). Conflit : deux règles pour la même case ■ **la grammaire n'est pas LL(1)**. C'est exactement l'ambiguïté classique du "dangling else" (on ne sait pas, en lisant un « e » (else), s'il faut le rattacher au S courant ou le laisser au niveau englobant).

Exercice 6

Correction

Entrée : (id + id) * id \$ — trace résumée (mêmes règles que la table de la section 6) :

Pile	Entrée	Action
E \$	(id+id)*id\$	E -> T E'
T E' \$	(id+id)*id\$	T -> F T'
F T' E' \$	(id+id)*id\$	F -> (E)
(E) T' E' \$	(id+id)*id\$	match (
E) T' E' \$	id+id)*id\$	E -> T E'
T E') T' E' \$	id+id)*id\$	T -> F T'
F T' E') T' E' \$	id+id)*id\$	F -> id ; match id
T' E') T' E' \$	+id)*id\$	T' -> eps
E') T' E' \$	+id)*id\$	E' -> + T E'
+ T E') T' E' \$	+id)*id\$	match + ; T -> F T' ; F->id ; match id
T' E') T' E' \$	)id\$	T' -> eps ; E' -> eps
) T' E' \$	)id\$	match)
T' E' \$	*id\$	T' -> * F T'
* F T' E' \$	*id\$	match * ; F -> id ; match id
T' E' \$	\$	T' -> eps ; E' -> eps
\$	\$	ACCEPTE